

Lando SSL Version 2 Well-formedness

January 19, 2022

1 Notation

Lists of t are written $[t]$; $[]_t$ is an empty list of t 's, $x :: xs$ is the list with head x and tail xs , and $[x]$ is the singleton list containing x . $last(l)$ returns the last element of (necessarily non-empty) list l . An optional t is written $t?$; a present option value x is written $[x]$ and a missing option value of type t is written $\cdot_t$. We write $x.y$ for the projection of field y from a node x . In the pattern $x@Foo\{\dots\}$, x is bound to the entire node labeled `Foo`. The pattern $(Foo|Bar)\{\dots\}$ matches nodes labeled either `Foo` or `Bar`. Disjoint union of maps is written $\uplus$, and is undefined if the domains of the maps are not disjoint. If m_1 and m_2 are maps then $m_1 \triangleleft m_2$ (pronounced “ m_1 shadows m_2 ”) is $m_1 \cup \{(k, m_2(k)) | k \in Dom(m_2) - Dom(m_1)\}$. An empty map is written $\emptyset$. A judgement applies only if all premises are fully defined.

2 Raw AST Grammar

This is intended to correspond precisely to the `RawAST` structure built by the current parser, omitting source position tags and comments. The AST corresponds directly to the concrete syntax, with one exception: implicit bodies of `systems` and `subsystems` are converted into explicit ones.

$s \in source$	$\rightarrow$	<code>Source{elems : [elem]}</code>
$e \in elem$	$\rightarrow$	<code>System{name : name, abbrev : name?, explanation : text, indexings : [indexing], body : [elem]}</code>
	$\rightarrow$	<code>Subsystem{name : name, abbrev : name?, inherits : [qname], clientOf : [qname], explanation : text, indexings : [indexing], body : [elem]}</code>
	$\rightarrow$	<code>SubsystemImport{name : qname, abbrev : name?, clientOf : [qname]}</code>
	$\rightarrow$	<code>Component{name : name, abbrev : name?, inherits : [qname], clientOf : [qname], explanation : text, parts : [part]}</code>
	$\rightarrow$	<code>ComponentImport{name : qname, abbrev : name?, clientOf : [qname]}</code>
	$\rightarrow$	<code>Events{name : name, events : [item]}</code>
	$\rightarrow$	<code>Scenarios{name : name, scenarios : [item]}</code>
	$\rightarrow$	<code>Requirements{name : name, requirements : [item]}</code>
	$\rightarrow$	<code>Relation{name : qname, inherits : [qname], clientOf : [qname]}</code>
$x \in indexing$	$\rightarrow$	<code>Indexing{key : text, values : [text]}</code>
$p \in part$	$\rightarrow$	<code>Constraint{text : text}</code>
	$\rightarrow$	<code>Query{text : text}</code>
	$\rightarrow$	<code>Command{text : text}</code>
$i \in item$	$\rightarrow$	<code>Item{id : name, text : text}</code>
$u \in uid$		(base type)
$t \in text$		(base type)
$n \in name$		(base type)
$q \in qname$	$=$	<code>[name]</code>
$c \in comment$		(base type)

3 Well-formedness Judgements

Although the AST does not explicitly include element labels, we assume that each node (in particular each *elem*) has a unique identity with a well-defined notion of equality.

$\Gamma : \text{name} \rightarrow \text{elem}$ is an environment (finite map) from names to their corresponding elements.

$\Phi : \text{elem} \rightarrow \Gamma$ is a finite map from (sub)systems to the environments they generate.

The function $qlook_\Gamma(q)$ resolved a qualified name q starting in environment Γ ; it is defined thus:

$$\begin{aligned} qlook_\Gamma([n]) &= \Gamma(n) \\ qlook_\Gamma(n :: ns) &= qlook_{\Phi(\Gamma(n))}(ns) \end{aligned}$$

$I : \text{elem} \times \text{elem}$ is an inheritance relation between elements, where $(e_1, e_2) \in I$ means e_1 inherits from e_2 . The predicate $nocycles(I)$ holds when I has no cycles.

We assume the existence of a function $qnames(t)$ that returns the list of *qnames* mentioned in *text* t .

A specification s is well-formed if it is possible to find a top-level environment Γ_0 , a global Φ_0 , and a global inheritance relation I_0 (all implicitly threaded everywhere) such that $\vdash s$.

$$\begin{array}{c} \frac{\Gamma_0 \vdash es \Rightarrow \Gamma_0 \quad nocycles(I) \quad \forall e_1, e_2 \in es, e_1 = \text{System}\{\dots\} \wedge e_2 = \text{System}\{\dots\} \implies e_1 = e_2 \quad \forall e \in es, \text{valid-toplevel}(e)}{\vdash \text{Source}\{\text{elems} = es\}} \\[10pt] \frac{\Gamma \vdash e \Rightarrow \Gamma' \quad \Gamma \vdash es \Rightarrow \Gamma''}{\Gamma \vdash e :: es \Rightarrow \Gamma' \uplus \Gamma''} \quad \frac{}{\Gamma \vdash []_{elem} \Rightarrow \{\}} \\[10pt] \frac{n \neq n_a \quad \Gamma \vdash t \quad \forall e \in es_b, \text{valid-contains}(e_s, e) \quad \Phi_0(e_s) = \Gamma' \quad \Gamma' \triangleleft \Gamma \vdash es_b \Rightarrow \Gamma'}{\Gamma \vdash e_s @ \text{System}\{\text{name} = n, \text{abbrev} = [n_a], \text{explanation} = t, \text{body} = es_b, \dots\} \Rightarrow \{n \mapsto e_s, n_a \mapsto e_s\}} \\[10pt] \frac{\Gamma \vdash t \quad \forall e \in es_b, \text{valid-contains}(e_s, e) \quad \Phi_0(e_s) = \Gamma' \quad \Gamma' \triangleleft \Gamma \vdash es_b \Rightarrow \Gamma'}{\Gamma \vdash e_s @ \text{System}\{\text{name} = n, \text{abbrev} = \cdot_{name}, \text{explanation} = t, \text{body} = es_b, \dots\} \Rightarrow \{n \mapsto e_s\}} \\[10pt] \frac{n \neq n_a \quad \forall q \in qs_i, \text{valid-inherit}(e_s, qlook_\Gamma(q)) \quad \forall q \in qs_c, \text{valid-client}(e_s, qlook_\Gamma(q)) \quad \Gamma \vdash t \quad \forall e \in es_b, \text{valid-contains}(e_s, e) \quad \Phi_0(e_s) = \Gamma' \quad \Gamma' \triangleleft \Gamma \vdash es_b \Rightarrow \Gamma'}{\Gamma \vdash e_s @ \text{Subsystem} \left\{ \begin{array}{l} \text{name} = n, \text{abbrev} = [n_a], \text{inherits} = qs_i, \\ \text{clientOf} = qs_c, \text{explanation} = t, \text{body} = es_b, \dots \end{array} \right\} \Rightarrow \{n \mapsto e_s, n_a \mapsto e_s\}} \\[10pt] \frac{\forall q \in qs_i, \text{valid-inherit}(e_s, qlook_\Gamma(q)) \quad \forall q \in qs_c, \text{valid-client}(e_s, qlook_\Gamma(q)) \quad \Gamma \vdash t \quad \forall e \in es_b, \text{valid-contains}(e_s, e) \quad \Phi_0(e_s) = \Gamma' \quad \Gamma' \triangleleft \Gamma \vdash es_b \Rightarrow \Gamma'}{\Gamma \vdash e_s @ \text{Subsystem} \left\{ \begin{array}{l} \text{name} = n, \text{abbrev} = \cdot_{name}, \text{inherits} = qs_i, \\ \text{clientOf} = qs_c, \text{explanation} = t, \text{body} = es_b, \dots \end{array} \right\} \Rightarrow \{n \mapsto e_s\}} \\[10pt] \frac{qlook_{\Gamma_0}(q) = e \quad \forall q \in qs_c, \text{valid-client}(e, qlook_\Gamma(q))}{\Gamma \vdash e_s @ \text{SubsystemImport}\{\text{name} = q, \text{abbrev} = [n_a], \text{clientOf} = qs_c\} \Rightarrow \{n_a \mapsto e_s\}} \\[10pt] \frac{qlook_{\Gamma_0}(q) = e \quad \forall q \in qs_c, \text{valid-client}(e, qlook_\Gamma(q))}{\Gamma \vdash e_s @ \text{SubsystemImport}\{\text{name} = q, \text{abbrev} = \cdot_{name}, \text{clientOf} = qs_c\} \Rightarrow \{last(q) \mapsto e_s\}} \\[10pt] \frac{n \neq n_a \quad \forall q \in qs_i, \text{valid-inherit}(e_c, qlook_\Gamma(q)) \quad \forall q \in qs_c, \text{valid-client}(e_c, qlook_\Gamma(q)) \quad \Gamma \vdash t \quad \forall p \in ps, \Gamma \vdash p}{\Gamma \vdash e_c @ \text{Component} \left\{ \begin{array}{l} \text{name} = n, \text{abbrev} = [n_a], \text{inherits} = qs_i, \\ \text{clientOf} = qs_c, \text{explanation} = t, \text{parts} = ps \end{array} \right\} \Rightarrow \{n \mapsto e_c, n_a \mapsto e_c\}} \end{array}$$

$$\begin{array}{c}
\frac{\forall q \in qs_i, \text{valid-inherit}(e_c, \text{qlook}_\Gamma(q)) \quad \forall q \in qs_c, \text{valid-client}(e_c, \text{qlook}_\Gamma(q)) \quad \Gamma \vdash t \quad \forall p \in ps, \Gamma \vdash p}{\Gamma \vdash e_c @ \text{Component} \left\{ \begin{array}{l} \text{name} = n, \text{abbrev} = \cdot_{name}, \text{inherits} = qs_i, \\ \text{clientOf} = qs_c, \text{explanation} = t, \text{parts} = ps \end{array} \right\} \Rightarrow \{n \mapsto e_c\}} \\
\\
\frac{\Gamma \vdash t}{\Gamma \vdash \text{Constraint}\{\text{text} = t\}} \quad \frac{\Gamma \vdash t}{\Gamma \vdash \text{Query}\{\text{text} = t\}} \quad \frac{\Gamma \vdash t}{\Gamma \vdash \text{Command}\{\text{text} = t\}} \\
\\
\frac{\text{qlook}_{\Gamma_0}(q) = e \quad \forall q \in qs_c, \text{valid-client}(e, \text{qlook}_\Gamma(q))}{\Gamma \vdash e_c @ \text{ComponentImport}\{\text{name} = q, \text{abbrev} = \lceil n_a \rceil, \text{clientOf} = qs_c\} \Rightarrow \{n_a \mapsto e_c\}} \\
\\
\frac{\text{qlook}_{\Gamma_0}(q) = e \quad \forall q \in qs_c, \text{valid-client}(e, \text{qlook}_\Gamma(q))}{\Gamma \vdash e_c @ \text{ComponentImport}\{\text{name} = q, \text{abbrev} = \cdot_{name}, \text{clientOf} = qs_c\} \Rightarrow \{last(q) \mapsto e_c\}} \\
\\
\frac{\Gamma \vdash is \Rightarrow \Gamma'}{\Gamma \vdash e @ \text{Events}\{\text{name} = n, \text{events} = is\} \Rightarrow \{n \mapsto e\} \uplus \Gamma'} \\
\\
\frac{\Gamma \vdash is \Rightarrow \Gamma'}{\Gamma \vdash e @ \text{Scenarios}\{\text{name} = n, \text{scenarios} = is\} \Rightarrow \{n \mapsto e\} \uplus \Gamma'} \\
\\
\frac{\Gamma \vdash is \Rightarrow \Gamma'}{\Gamma \vdash e @ \text{Requirements}\{\text{name} = n, \text{requirements} = is\} \Rightarrow \{n \mapsto e\} \uplus \Gamma'} \quad \frac{}{\Gamma \vdash \llbracket item \rrbracket \Rightarrow \{\}} \\
\\
\frac{\Gamma \vdash i \Rightarrow \Gamma'' \quad \Gamma \vdash is \Rightarrow \Gamma'''}{\Gamma \vdash i :: is \Rightarrow \Gamma'' \uplus \Gamma'''} \quad \frac{\Gamma \vdash t}{\Gamma \vdash i @ \text{Item}\{\text{id} = n, \text{text} = t\} \Rightarrow \{n \mapsto i\}} \\
\\
\frac{\text{qlook}_\Gamma(q) = e \quad \forall q \in qs_i, \text{valid-inherit}(e, \text{qlook}_\Gamma(q)) \quad \forall q \in qs_c, \text{valid-client}(e, \text{qlook}_\Gamma(q))}{\Gamma \vdash \text{Relation}\{\text{name} = q, \text{inherits} = qs_i, \text{clientOf} = qs_c\} \Rightarrow \{\}} \\
\\
\frac{\forall n_t \in qnames(t), \exists e, \Gamma(n_t) = e}{\Gamma \vdash t} \\
\\
\text{valid-toplevel}((\text{System} | \text{Subsystem} | \text{Component} | \text{Events} | \text{Scenarios} | \text{Requirements} | \text{Relation})\{\dots\}) \\
\\
\frac{}{\text{valid-contains}(\text{System}\{\dots\}, (\text{Subsystem} | \text{SubsystemImport} | \text{Relation})\{\dots\})} \\
\\
\frac{}{\text{valid-contains}(\text{Subsystem}\{\dots\}, (\text{Subsystem} | \text{SubsystemImport} | \text{Component} | \text{ComponentImport})\{\dots\})} \\
\\
\frac{}{\text{valid-contains}(\text{Subsystem}\{\dots\}, (\text{Scenarios} | \text{Requirements} | \text{Events} | \text{Relation})\{\dots\})} \\
\\
\frac{(e_1, e_2) \in I_0}{\text{valid-inherit}(e_1 @ \text{Component}\{\dots\}, e_2 @ \text{Component}\{\dots\})} \\
\\
\text{valid-client}((\text{Subsystem} | \text{SubsystemImport})\{\dots\}, (\text{Subsystem} | \text{SubsystemImport} | \text{Component} | \text{ComponentImport})\{\dots\}) \\
\\
\text{valid-client}((\text{Component} | \text{ComponentImport})\{\dots\}, (\text{Subsystem} | \text{SubsystemImport} | \text{Component} | \text{ComponentImport})\{\dots\})
\end{array}$$

